

PixSearch

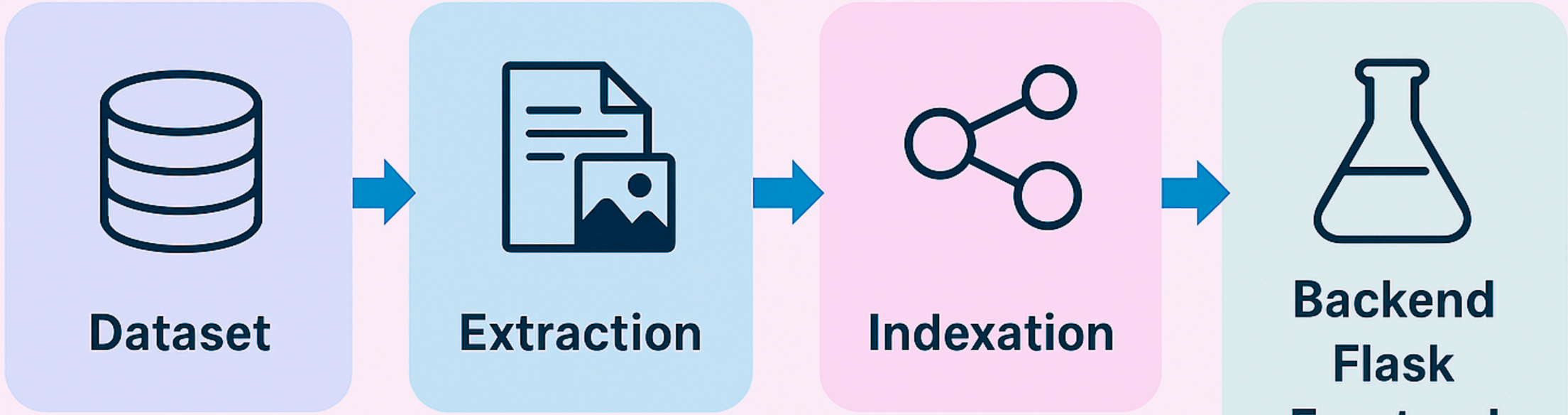
SARRA MAKHLOUF

AVRIL 2025

Introduction

PixSearch est un moteur de recherche intelligent d’images, combinant analyse textuelle (TF-IDF) et visuelle (MobileNet) sur un corpus provenant de Dataset Unsplash.
Il permet une exploration intuitive et rapide grâce à l’indexation Elasticsearch.

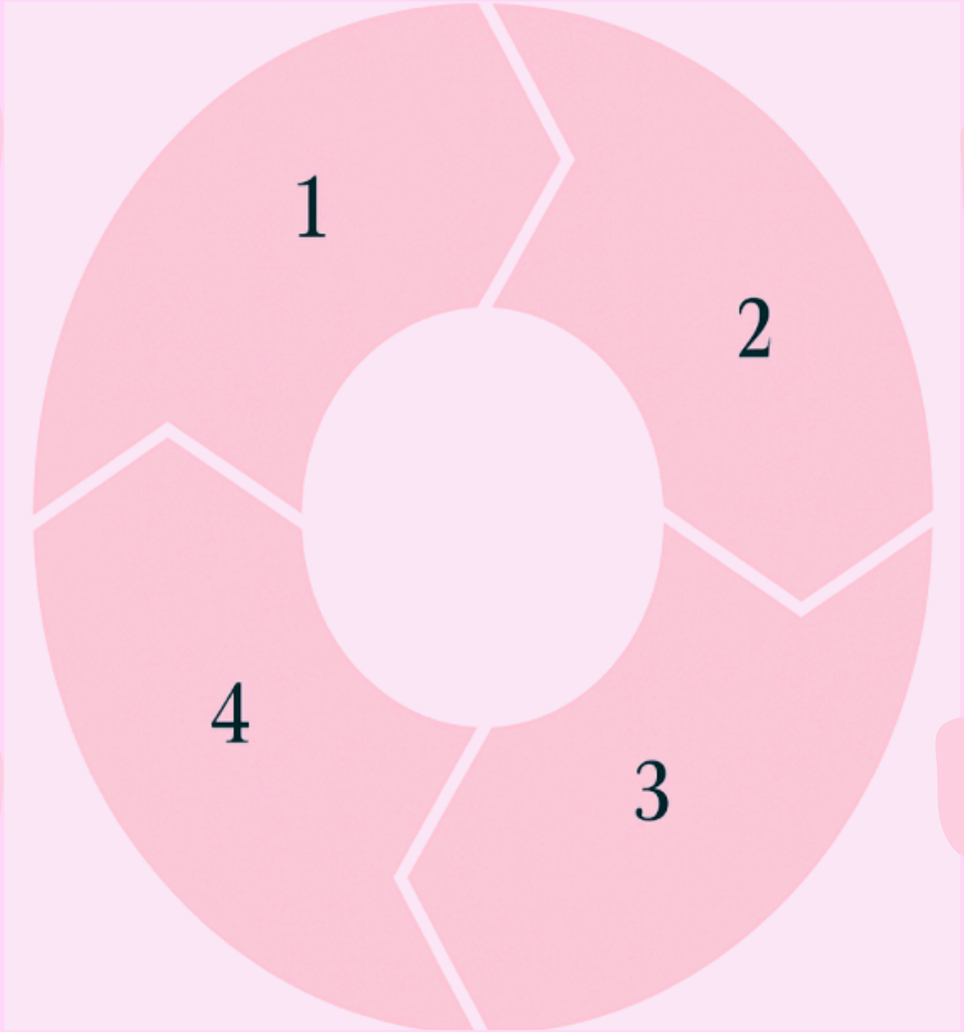
Architecture de Projet



1.Préparation des données

chargement

nettoyage



Sauvegarde

Préparation

Techniques utilisées

```
import pandas as pd
import os
import json
```

Outil	***Rôle***
pandas	Lire les fichiers TSV (photos, keywords, colors) et les manipuler facilement.
os	Gérer les chemins vers les fichiers.
json	Sauvegarder les données préparées dans un format JSON pour l’indexation Elasticsearch.

Exemple:

```
self.datasets['photos'] = pd.read_csv(
    os.path.join(self.data_path, "photos.tsv000"),
    sep='\t',
    header=0,
    usecols=['photo_id', 'photo_image_url', 'photo_description', 'stats_views']
)
```

```
self.datasets['photos']['photo_description'] = (
    self.datasets['photos']['photo_description']
    .fillna("No description")
    .str[:500]  # Limite la taille
)
# Filtre les photos sans image_url
self.datasets['photos'] = self.datasets['photos'].dropna(subset=['photo_image_url'])
print("Données nettoyées")
```

On a constitué la base d’extraction des données brutes du dataset Unsplash. Grâce à pandas, les fichiers .tsv sont lus, nettoyés et enrichis (mots-clés, descriptions, couleurs), puis convertis en une structure JSON exploitable pour l’indexation dans Elasticsearch.

2.Extraction des caractéristiques textuelles

1/ Prétraitement du texte : concatène description + mots-clés, transforme en minuscules.

```
 # Concatène description + keywords
# Nettoyage du texte avant vectorisation
 text_data = [
     f"{doc.get('description', '')} {' '.join(doc.get('keywords', []))}".lower()
     for doc in documents
 ]
```

2/ Vectorization TF-IDF :

- Capture les bigrammes pour améliorer la compréhension contextuelle
- Ignore les mots trop rares

Techniques utilisées

```
import pickle
from sklearn.feature_extraction.text import TfidfVectorizer
```

Outil	***Rôle***
scikit-learn (TfidfVectorizer)	Transformer les textes (description + mots-clés) en vecteurs numériques (TF-IDF) pour mesurer la similarité.
pickle	Sauvegarder le modèle TF-IDF entraîné pour réutilisation ultérieure.

```python
# TF-IDF vectorisation
vectorizer = TfidfVectorizer(
    stop_words='english',
    max_features=4000,
    ngram_range=(1, 2),  # Capture les bigrammes
    min_df=3  # Ignore les termes trop rares
)
# Gestion mémoire optimale
vectors = vectorizer.fit_transform(text_data)
```

3/ Sauvegarde :

- **text_vectorizer.pk** : le modéle vectoriseur
- **text_vectors.npz** : les vecteurs textuelles
- **text_features.json** : les mots retenus par le modèle

On a convertit les textes en vecteurs numériques à l’aide de la technique TF-IDF (Term Frequency – Inverse Document Frequency). Cela permet de quantifier l’importance des mots pour chaque image

3.Extraction des caractéristiques visuelles (images)

Techniques utilisées

```
import torch
import torch.nn as nn
from torchvision import models, transforms
import requests
import numpy as np
from tqdm import tqdm
import json
from PIL import Image
from io import BytesIO
import random
from ultralytics import YOLO
```

Outil	***Rôle***
torch , torchvision	Chargement et utilisation de MobileNetV3 pour extraire des features visuelles.
PIL, requests	Téléchargement et traitement d’images depuis les URLs.
tqdm	Suivi de la progression du traitement par lot.
numpy , json	Traitement et sauvegarde des vecteurs
io.BytesIO	Permet de lire les images directement depuis les données binaires téléchargées via requests.

MobileNetV3 (extraction de caractéristiques):

C’est un réseau neuronal léger basé sur des blocs Mobile Inverted Bottleneck avec attention (SE blocks), conçu pour extraire efficacement des vecteurs d’image sur des appareils à ressources limitées.

YOLOv8(détection d’objets):

C’est un modèle de détection d’objets rapide et précis, basé sur une architecture anchor-free, idéal pour localiser et classer plusieurs objets dans une image en temps réel

Sert ici à localiser les objets dans l’image avant de les passer à MobileNet pour l’extraction vectorielle.

Outil:

```
from ultralytics import YOLO
```

Initialisation du modèle MobileNetV3

```
# MobileNet
mobilenet = models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.DEFAULT)
mobilenet.classifier[-1] = nn.Identity()
mobilenet = mobilenet.to(device).eval()
```

Initialisation du YOLOv8

```
# YOLOv8 (utilise le modèle léger pour rapidité plus que YOLOv5)
yolo = YOLO("yolov8n.pt")
```

Préprocessing MoblieNet

```
# Préprocessing MobileNet
preprocess = transforms.Compose([
    transforms.Resize((224, 224)),  # Adapter à la crop
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
```

Détection des Objets:

```python
def extract_objects(image):
    results = yolo(image, device='cpu', verbose=False)

    detected = []
    for result in results:
        for box in result.boxes:
            try:
                x1, y1, x2, y2 = map(int, box.xyxy[0])
                class_id = int(box.cls[0])
                label = result.names[class_id]
                # Découpe de l'objet détecté à partir des coordonnées (bounding box) avec PIL
                cropped = image.crop((x1, y1, x2, y2))
                tensor = preprocess(cropped).unsqueeze(0).to(device)
                with torch.no_grad():
                    vector = mobilenet(tensor).cpu().numpy().flatten().tolist()
                detected.append({
                    "label": label,
                    "vector": vector
                })
```

on a combiné deux modèles complémentaires de vision par ordinateur pré-entraînés :

YOLOv8 est utilisé pour détecter et localiser automatiquement les objets présents dans chaque image à l’aide de bounding boxes

MobileNetV3 un modèle léger et efficace, pour extraire un vecteur visuel décrivant son apparence

Cette collaboration permet d’obtenir une représentation fine et segmentée de l’image, ce qui améliore considérablement la pertinence des résultats de recherche.

4.Indexation dans ElasticSearch

Objectif :

Rendre les documents accessibles et exploitables par un moteur de recherche via Elasticsearch, en indexant à la fois des métadonnées textuelles et des liens vers les vecteurs d’image.

Techniques utilisées

```
from elasticsearch import Elasticsearch, helpers
import json
import os
from pathlib import Path
```

Outil	***Rôle***
ElasticSearch	Base d'indexation et de recherche
elasticsearch.helpers	Import massif de documents
json, pathlib	Lecture et validation des données
os	Vérification de la présence de fichiers

Connexion et Configuration

```python
# 1. Configuration
DATASET_PATH = Path("output_vectors.json")  # Chemin vers le fichier de données (issu de extract_imag
ES_HOST = "http://localhost:9200"
INDEX_NAME = "unsplash_final"  # Nouveau nom d'index

# 2. Connexion Elasticsearch
es = Elasticsearch(ES_HOST)
```


```
"name" : "DESKTOP-K2EUF00",
"cluster_name" : "elasticsearch",
"cluster_uuid" : "kXjwzRW7QSSyizdaF6pkXA",
"version" : {
  "number" : "8.17.4",
  "build_flavor" : "default",
  "build_type" : "zip",
  "build_hash" : "c63c7f5f8ce7d2e4805b7b3d842e7e792d84dda1",
  "build_date" : "2025-03-20T15:39:59.811110136Z",
  "build_snapshot" : false,
  "lucene_version" : "9.12.0",
  "minimum_wire_compatibility_version" : "7.17.0",
  "minimum_index_compatibility_version" : "7.0.0"
},
"tagline" : "You Know, for Search"
```

Définition d’un mapping personnalisé :

```
mapping = {
        "settings": {
            "number_of_shards": 1,
            "analysis": {
                "filter": {
                    "french_elision": {
                        "type": "elision",
                        "articles": ["l", "m", "t", "qu", "n", "s", "j", "d"]
                    }
                },
                "analyzer": {
                    "french": {
                        "tokenizer": "standard",
                        "filter": ["lowercase", "french_elision", "asciifolding"]
                    }
                }
            }
        },
        "mappings": {
            "properties": {
                "description": {"type": "text", "analyzer": "french"},
                "keywords": {"type": "keyword"},
                "keywords_suggest": {"type": "completion"},  # champ pour les suggestions
                "image_url": {"type": "keyword"},
                "views": {"type": "integer"}
            }
```

Création d’Index :

```
if es.indices.exists(index=INDEX_NAME):
    es.indices.delete(index=INDEX_NAME)

es.indices.create(index=INDEX_NAME, body=mapping)
print(f"Index {INDEX_NAME} créé avec succès")
return True
```

Indexation optimisée :

```
success, _ = helpers.bulk(es, actions, chunk_size=100)
print(f"\nIndexation réussie: {success}/{len(documents)} documents")
print(f"Accéder aux données: http://localhost:9200/{INDEX_NAME}/_search")
```

Tous les documents enrichis sont indexés dans l’index **unsplash_final**

Indexation des documents :

```python
def index_documents(documents):
    """Indexation avec nettoyage des vecteurs"""
    actions = []
    for doc in documents:
        # Validation des champs obligatoires
        if not all(k in doc for k in ["description", "keywords", "url"]):
            continue

        # Nettoyage des objets détectés
        valid_objects = []
        for obj in doc.get("detected_objects", []):
            vector = obj.get("vector", [])
            if (
                isinstance(vector, list) and
                len(vector) == 576 and
                all(isinstance(x, (float, int)) for x in vector)
            ):
                valid_objects.append(obj)

        # Ajoute les objets valides au document
        doc["detected_objects"] = valid_objects

        actions.append({
            "_index": INDEX_NAME,
            "_source": {
                "description": doc.get("description", ""),
                "keywords": doc.get("keywords", []),
                "keywords_suggest": {
                    "input": doc.get("keywords", []),
                    "weight": doc.get("views", 0)
                },
                "image_url": doc.get("url", ""),
                "views": doc.get("views", 0),
            }
        })
```

5.Interface Web avec Flask

Objectif

- Rechercher des images par mots-clés.
- Recevoir des suggestions dynamiques via auto-complétion.
- visualiser les résultats de façon claire et interactive.

Techniques utilisées

```
from flask import Flask, request, render_template, jsonify
from elasticsearch import Elasticsearch
```

Outil	*Rôle*
Flask	Framework web léger
Jinja2	Moteur de template pour les vues HTMLs
Elasticsearch	Backend de recherche
HTML/CSS/JS	Frontend dans search.html et results.html

```python
#Initialise l'app Flask
app = Flask(__name__)

#configuraton d'un filtre Jinja2 pour formater les nombres
Tabnine | Edit | Test | Explain | Document
@app.template_filter('format_number')
def format_number_filter(value):
    return f"{value:,}".replace(",", " ")

#Connexion à elasticSearch
es = Elasticsearch("http://localhost:9200")
INDEX_NAME = "unsplash_final"
```

Les routes :

Route / : C’est le point d’entrée de l’application pour l’utilisateur.

```python
@app.route('/')
def home():
    return render_template('search.html')
```

/search : Permet de rechercher des images en plusieurs langues et avec des erreurs de frappe mineures.

```python
@app.route('/search')
def search():
```

```python
    query = request.args.get('q', '').lower()
    translated_query = TRANSLATIONS.get(query, query)
```

Ce corps de requête Elasticsearch permet de retrouver les images les plus pertinentes en combinant les champs textuels associés aux objets détectés et leur description, avec un système de pondération et de tolérance aux fautes.

```
body = {
"query": {
    "bool": {
        "should": [
            {
                "match": {
                    "keywords": {
                        "query": translated_query,
                        "boost": 5
                    }
                }
            },
            {
                "match": {
                    "description": {
                        "query": translated_query,
                        "boost": 2 ,
                        "fuzziness": "AUTO"
                    }
                }
            },
            {
                "match": {
                    "alt_description": {
                        "query": translated_query,
                        "boost": 1 ,
                        "fuzziness": "AUTO"
                    }
                }
            }
        ],
```

/suggest : Améliore l’expérience utilisateur grâce à une autocomplétion rapide et pertinente.

```python
@app.route('/suggest')
def suggest():
    prefix = request.args.get('q', '').lower()

    if len(prefix) < 2:
        return jsonify([])

    body = {
        "suggest": {
            "keyword-suggest": {
                "prefix": prefix,
                "completion": {
                    "field": "keywords_suggest",
                    "skip_duplicates": True,
                    "size": 5
                }
            }
        }
```

Conclusion

Ce projet démontre la mise en place efficace d’un moteur de recherche intelligent basé sur un dataset d’images (Unsplash), combinant :

- Une recherche textuelle performante avec fuzziness et analyse linguistique
- Un système de suggestions dynamiques pour améliorer l'expérience utilisateur,
- Une interface web simple et intuitive développée avec Flask,
- Une indexation structurée des données via Elasticsearch, optimisée pour la pertinence.

Merci Pour votre attention